

회귀 Regression — 숫자를 예측하는 첫 모델

내년 이맘때 이 아파트는 얼마일까 — '얼마나'를 맞추는 문제는 전부 회귀의 일이다

회귀는 흩어진 점들 사이로 가장 그럴듯한 직선 $y = wx + b$ 를 긋는 일입니다. 어느 선이 더 나은지는 **MSE**로 점수를 매기고, 그 점수가 낮아지는 쪽으로 **경사하강법**이 선을 조금씩 옮깁니다. 단서가 여러 개로 늘면 **다중 회귀**가 되고, 학습 데이터를 지나치게 외운 모델은 **규제(Lasso·Ridge)**로 잡습니다.

오늘 끝내면 할 수 있는 것

네 가지면 충분합니다.

01

직선을 세운다

독립변수와 종속변수를 구분하고, 가설 함수 $y = wx + b$ 에서 **w**와 **b**가 무엇을 뜻하는지 그림과 숫자로 설명할 수 있다.

02

채점하고 고친다

MSE로 선을 채점하고, **경사하강법**이 학습률 α 만큼 내려가며 $w \cdot b$ 를 고쳐가는 과정을 수식으로 풀 수 있다.

03

여러 단서로 넓힌다

다중 선형회귀로 변수를 여러 개 쓰고, 학습된 **계수**로 각 단서가 결과를 어느 쪽으로 미는지 읽을 수 있다.

04

외운 모델을 다스린다

과적합을 그림으로 알아보고, **Lasso(L1)**와 **Ridge(L2)**가 각각 계수를 어떻게 누르는지 구분해 쓸 수 있다.

내년 이맘때, 이 아파트는 얼마일까

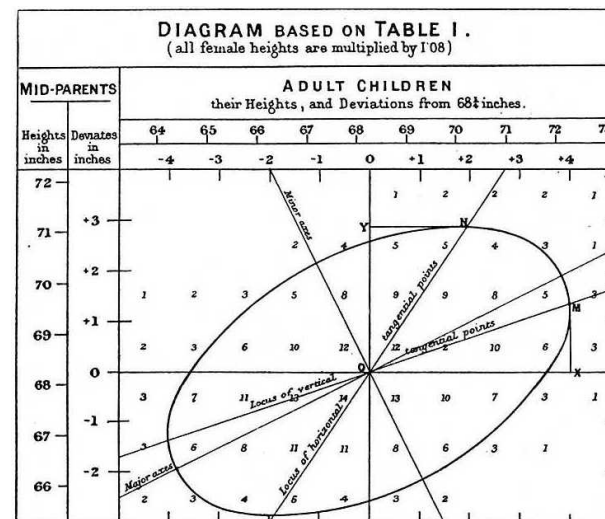
부동산 앱이 띄우는 '예상 시세 4억 2,300만 원' — 이 숫자는 어디서 왔을까요? **회귀**로 계산한 값입니다.

지도학습은 둘로 갈립니다. "스팸이냐 아니냐"처럼 **어느 쪽**인지 고르면 분류, "얼마냐"처럼 **연속된 숫자**를 맞히면 회귀입니다. 가격은 4억과 4억 1천 사이에도 값이 무한한 **연속값**이라 회귀 문제죠 — 내일 기온, 다음 달 매출도 한 식구입니다.

회귀(回歸)라는 이름의 유래. 19세기 통계학자 골턴이 부모와 자식 키를 모아 그린 **오른쪽 도표**에서 출발합니다. "키 큰 부모의 자식 키는 평균으로 **되돌아간다**"는 현상에 붙인 이름이, 지금은 '연속값 예측' 전반을 가리키게 됐어요.

방법은 의외로 소박합니다. 데이터를 좌표평면에 점으로 찍고, 점들 사이로 **가장 그럴듯한 선**을 하나 긋습니다.

선을 **어떻게 긋고**(§2), 단서가 여러 개면 **어떻게 늘리고**(§3), 지나치게 외울 때 **어떻게 잡는지**(§4)를 차례로 봅니다.



골턴의 원본 회귀 도표 — 부모 키(세로)와 자식 키(가로). 기울어진 타원과 두 회귀선이 '평균으로의 회귀'를 보여준다. 출처: F. Galton (1886), Public Domain, via Wikimedia Commons.

‘회귀’라는 말은 바로 이 그림에서 태어났다.

고르는 문제냐, 맞는 문제냐 — 분류 vs 회귀

지도학습의 두 갈래입니다. **출력이 무엇이나**만 보면 바로 구분돼요.

정해진 보기에서 고른다

분류 Classification

출력 = 라벨

스팸/정상, 고양이/개/새 — 답은 **미리 정해진 보기** 중 하나입니다. 보기와 보기 사이의 중간값은 존재하지 않아요.

이런 질문들

"이 메일은 스팸인가?", "이 사진 속 동물은?", "이 거래는 사기인가?" — 전부 **어느 쪽?**을 묻는 질문입니다.

틀림이 명확하다

고양이를 개라고 하면 그냥 오답입니다. 맞혔냐 틀렸냐, **맞힌 비율(정확도)**로 채점해요.

연속된 숫자를 맞힌다

회귀 Regression

출력 = 연속값

4억 2,300만 원, 23.7도 — 답이 **실수 축 위 어디든** 떨어질 수 있습니다. 무한히 많은 후보 중 한 점을 내놓아요.

이런 질문들

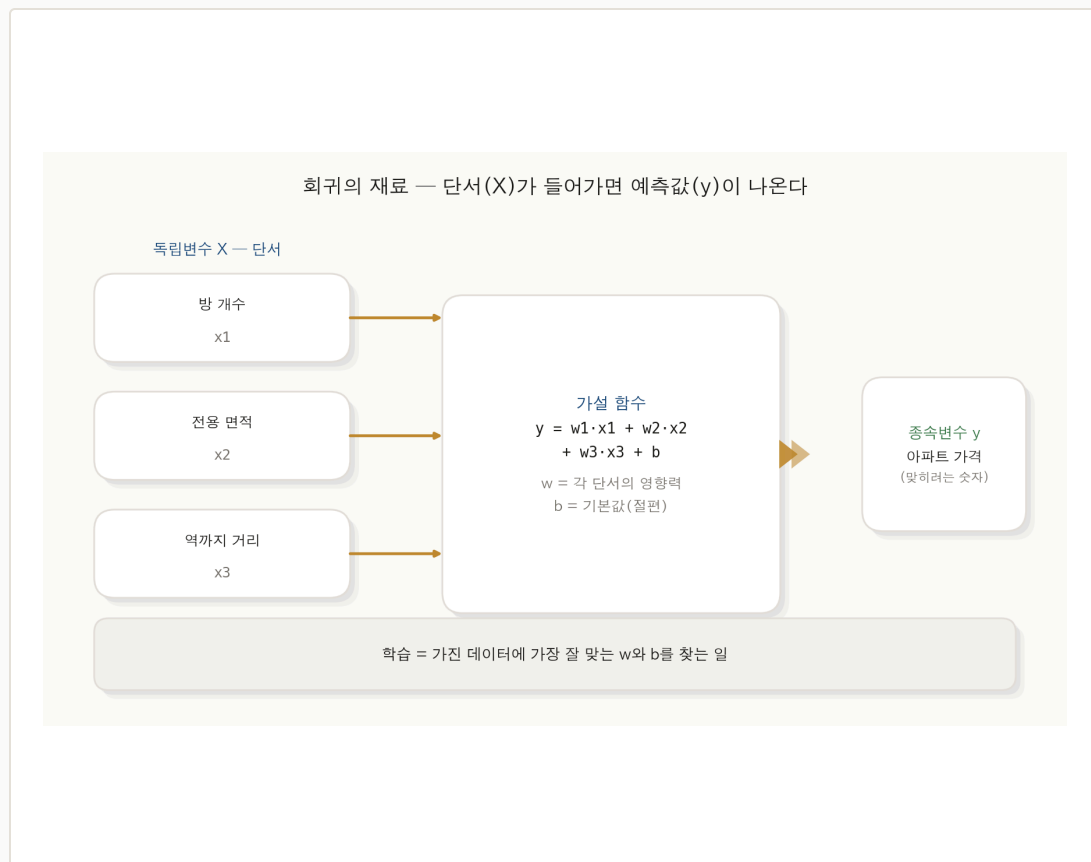
"이 집은 얼마?", "내일 기온은 몇 도?", "이 영화의 관객 수는?" — 전부 **얼마나?**를 묻는 질문입니다.

'얼마나' 틀렸는지 잴다

4억짜리를 4.1억이라 하면 조금, 8억이라 하면 크게 틀린 거죠. 그래서 **오차의 크기**로 채점합니다 — §2의 MSE가 그 자예요.

독립변수, 종속변수, 그리고 가설 함수

독립변수 X 는 단서(방 개수·면적), 종속변수 y 는 맞히려는 값(가격)입니다. 둘을 잇는 식 $y = w_1x_1 + w_2x_2 + b$ 가 **가설 함수** — 학습은 최적의 $w \cdot b$ 찾기예요.



01 독립변수 X — 단서

가격에 영향을 줄 후보들입니다. 방 개수, 면적, 역까지 거리 — 모델에 **입력**되는 쪽이에요.

02 종속변수 y — 과녁

독립변수에 **따라** 움직이는 값, 우리가 맞히려는 가격입니다. 데이터에선 정답 라벨로 주어져요.

03 회귀 계수 w — 영향력

w_1 이 크면 방 개수가 가격을 크게 움직인다는 뜻입니다. 학습이 찾아내는 **핵심 미지수**죠.

04 가설(hypothesis)인 이유

"관계가 이런 모양일 것"이라는 **추측**에서 출발하니까요. 데이터로 $w \cdot b$ 를 다듬어 가설을 완성합니다.

점 사이로 가장 그럴듯한 직선 하나 — $y = wx + b$



01 변수가 하나면 '단순' 선형회귀

면적 하나만으로 가격을 맞히는 가장 작은 회귀입니다. 2차원 평면이라 **눈으로 전 과정을 볼 수 있어서** 공부에 늘 여기서 시작돼요.

02 w — 기울기

x가 1 늘 때 y가 얼마나 변하는지입니다. $w = 0.3$ 이면 **1평 넓어질 때마다 3천만 원이** 붙는다는 뜻 — 숫자에 의미가 담깁니다.

03 b — 절편

x가 0일 때의 값, 선의 **시작 높이**입니다. b가 없으면 모든 선이 원점을 지나야 해서, 맞출 수 있는 모양이 확 줄어요.

04 잔차 — 선이 못 맞힌 만큼

각 점에서 선까지의 세로 거리, 즉 **실제값 - 예측값**입니다. 좋은 선이란 잔차들이 전체적으로 작은 선 — 그 '작다'를 숫자로 재는 법은 곧 보겠습니다.

면적-가격 산점도와 회귀선 — 점과 선의 세로 거리(잔차)가 곧 그 점에서의 오차다.

숫자를 넣어보면 손에 잡힌다 — 예측과 잔차

$y = wx + b$ 는 추상적이지만, 숫자를 한 번 대입해 보면 기울기와 절편의 의미가 또렷해져요. (가격 단위: 억 원)

기울기와 절편을 정합니다. 데이터로 학습된 선이 $w = 0.11$, $b = 0.7$ 이라 합시다. 그러면 예측식은 $\text{예측가격} = 0.11 \times \text{면적} + 0.7$ 이 됩니다 — §2.1 그림의 바로 그 선이에요.

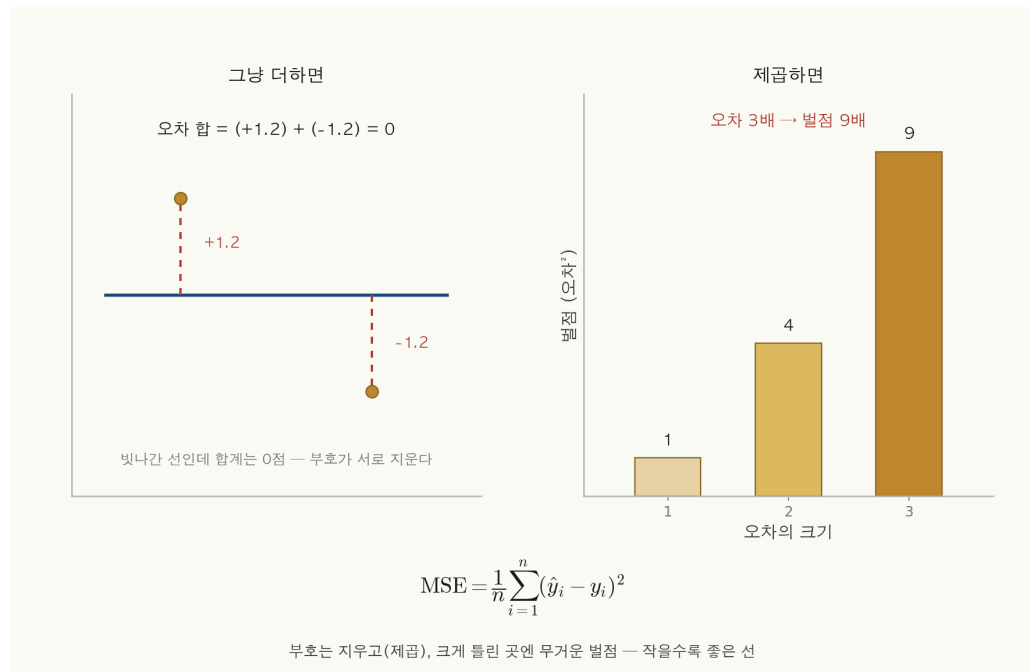
새 집을 넣어 예측합니다. 전용 면적 **20평**이면 $0.11 \times 20 + 0.7 = 2.9\text{억}$, **30평**이면 $0.11 \times 30 + 0.7 = 4.0\text{억}$. 면적이 10평 늘 때 가격이 $0.11 \times 10 = 1.1\text{억}$ 오릅니다 — 이 '1평당 0.11억'이 바로 기울기의 의미죠.

잔차로 정답과 비교합니다. 20평 집의 **실제** 거래가가 3.3억이었다면, 잔차 = 실제 - 예측 = $3.3 - 2.9 = +0.4\text{억}$. 선이 이 집을 0.4억 **낮게** 본 겁니다. 다른 집은 +0.4억, 또 -0.2억처럼 잔차가 제각각이에요.

그 잔차들을 한 숫자로 모으면 MSE. 잔차를 제곱해(부호를 지우고) 평균 내면 '이 선이 전체적으로 얼마나 틀렸나'가 점수 하나로 나옵니다. 학습이란 결국 **이 점수를 가장 작게** 만드는 $w \cdot b$ 를 찾는 일 — 다음 장에서 그 점수를 정의합니다.

오차를 왜 하필 제곱할까 — MSE

평균 제곱 오차 $MSE = (1/n) \sum (\text{실제} - \text{예측})^2$ — 모든 점의 오차를 제곱해 평균 낸 점수입니다. 작을수록 좋은 선이고, 학습은 이 값을 **낮추는 방향**으로 진행돼요.



01 그냥 더하면 사고가 난다

오차 +1.2와 -1.2를 더하면 0 — **빛나간 선이 만점**을 받습니다. 부호가 서로를 지워버리는 거죠.

02 큰 오차에 무거운 벌점

오차 3은 벌점 9, 오차 10은 벌점 100. 제곱은 **크게 틀린 점**부터 고치도록 모델을 압박해요.

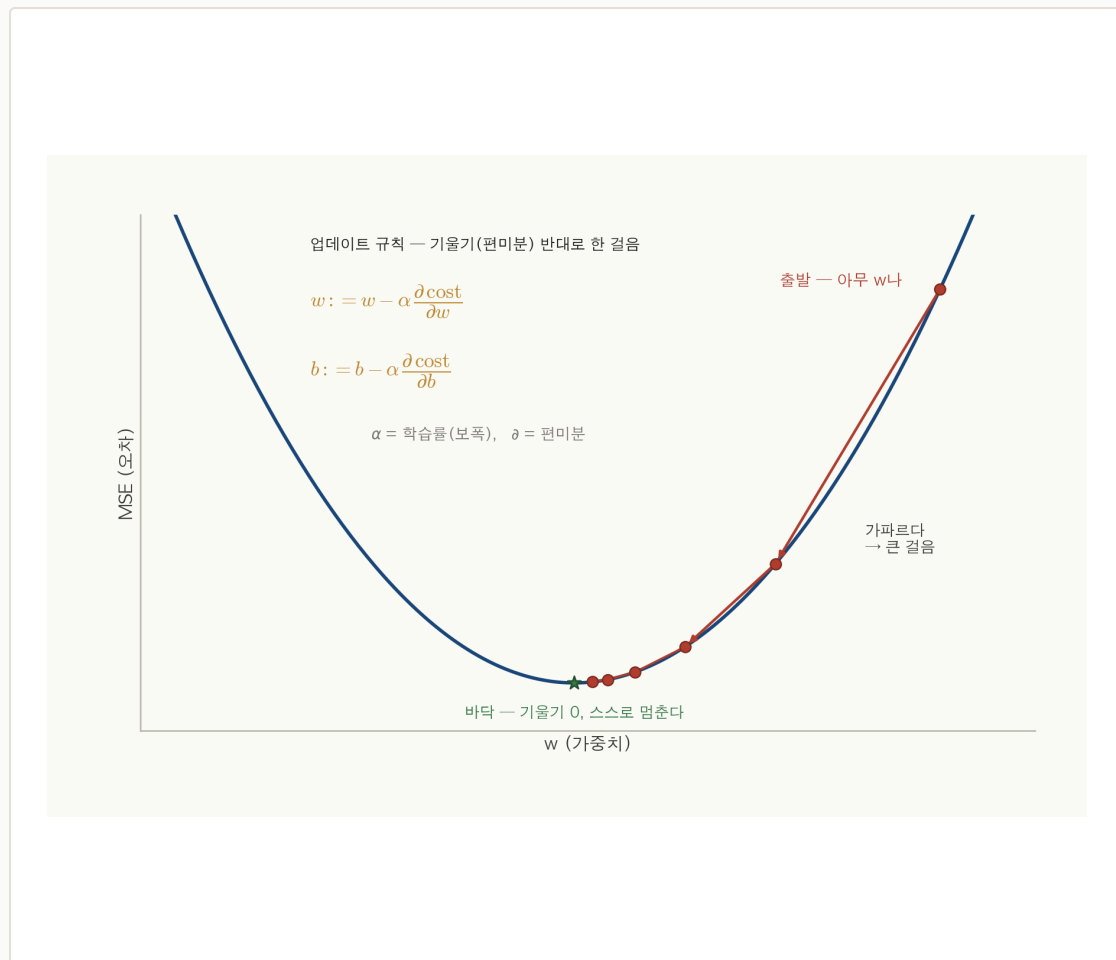
03 절댓값 대신 제곱인 이유

절댓값도 부호를 지우지만, 제곱엔 **덤이 있어요** — **어디서나 매끄럽게 미분**돼 경사하강법과 궁합이 좋습니다.

04 비용 함수라고 부른다

모델이 틀린 정도를 매기는 채점표라 cost/loss function이라 해요. **채점이 있어야 개선**이 있습니다.

경사하강법 — 기울기를 따라 골짜기로 내려간다



가로축 w , 세로축 MSE — 발밑 기울기를 따라 한 걸음씩 내려가면 골짜기(최소 오차)에 닿는다.

01 w 를 바꾸면 MSE가 변한다

w 값마다 MSE를 찍으면 **밥그릇 모양 곡선**이 됩니다. 좋은 w 찾기 = 이 그릇의 바닥 찾기 — 문제가 '지형 탐색'으로 바뀌어요.

02 안개 속이라 전체는 안 보인다

지형 전체를 다 계산해 보는 건 변수가 많아지면 불가능합니다. 보이는 건 **지금 서 있는 곳의 기울기** — 그래서 한 걸음씩 갑니다.

03 규칙은 한 줄

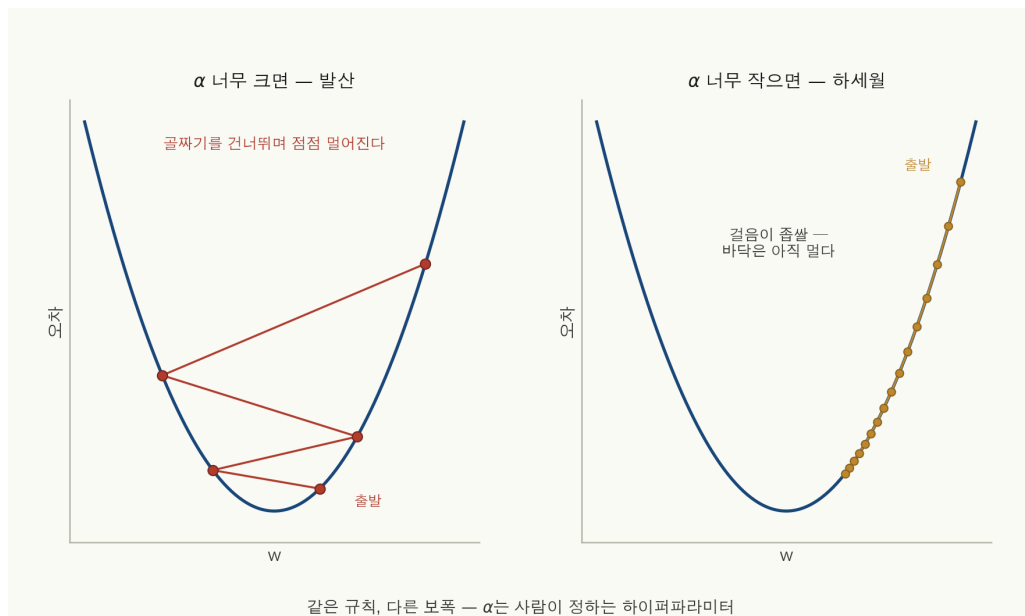
$w := w - \alpha \cdot \partial \text{cost} / \partial w$ (b도 같은 규칙) — 기울기, 즉 **편미분**의 반대 방향으로 α 만큼 이동합니다. 내리막이 가파르면 자동으로 큰 걸음, 완만하면 잔걸음이 돼요.

04 멈출 곳도 스스로 안다

바닥에선 기울기가 0이라 식이 **저절로 멈춥니다**. 예측 → 채점(MSE) → 한 걸음(업데이트) — 이 반복이 '학습'의 정체예요.

보폭 α — 너무 크면 추락, 너무 작으면 제자리

학습률(learning rate) α 는 한 걸음의 **보폭**입니다. 모델이 배우는 값이 아니라 사람이 정해주는 **하이퍼파라미터** — 이 하나를 잘못 잡으면 학습 전체가 어긋나요.



01 너무 크면 — 발산

골짜기를 한 번에 건너뛰고 반대편 **더 높은 곳**에 착지합니다. 오차가 줄긴커녕 튕기며 커져요.

02 너무 작으면 — 하세월

방향은 맞는데 걸음이 좁쌀만 합니다. 수만 번을 걸어도 바닥 근처엔 **못 가보고** 끝나요.

03 적당하면 — 성큼, 잔걸음

멀 땀 성큼, 바닥 근처에선 잔걸음. 보통 0.001~0.1 사이에서 **몇 가지를 실험**해 고릅니다.

04 기울기와 역할이 다르다

기울기는 **방향과 다급함**을, α 는 **그 말을 얼마나 믿을지**를 정합니다. 둘의 곱이 한 걸음이 돼요.

예측, 채점, 수정 — 세 박자의 반복

지금까지의 조각을 한 줄에 꿰면, 선형회귀의 학습은 이 세 박자를 **지칠 때까지 반복**하는 일입니다.

예측

지금의 $w \cdot b$ 로 일단 굿는다

현재 가설 $y = wx + b$ 로 모든 점의 예측값을 냅니다.
처음엔 $w \cdot b$ 가 무작위라 **형편없는 선**이어도 괜찮아요 — 어차피 출발점일 뿐입니다.

채점

MSE로 얼마나 틀렸나 잼다

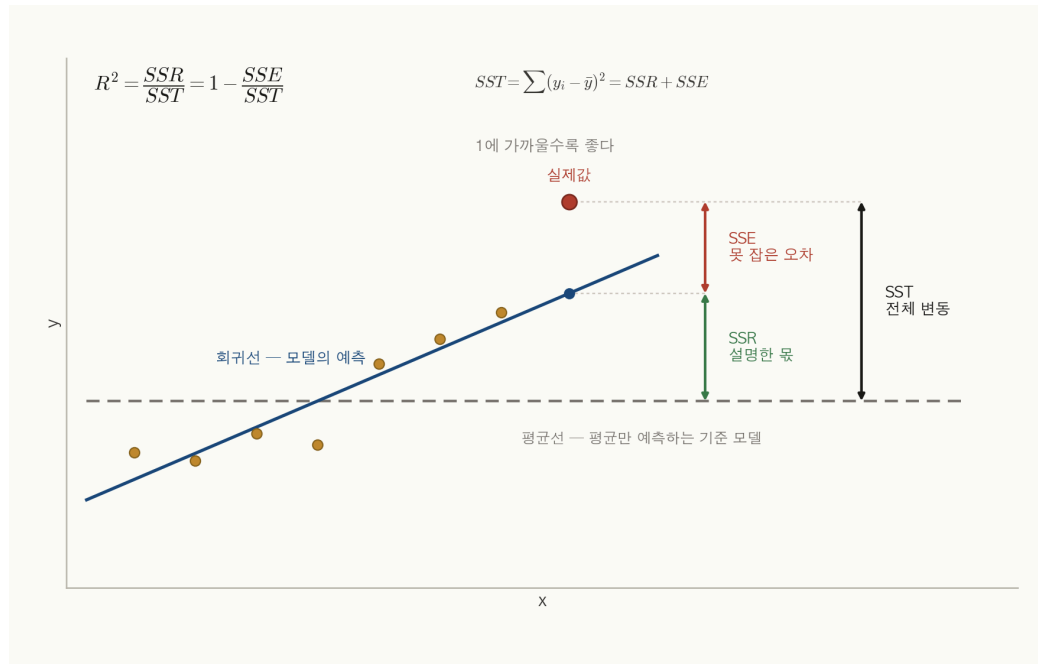
예측과 실제의 차이를 제공해 평균 냅니다. 이 점수가 곧 **지형의 높이** — 지금 선이 얼마나 나쁜지가 숫자 하나로 요약돼요.

수정

경사 반대로 한 걸음

$w := w - \alpha \cdot \partial \text{cost} / \partial w$ 로 w 와 b 를 살짝 고치고 다시 예측으로 돌아갑니다. 이 루프를 수백 번 돌면 — 점수가 더는 안 내려가는 곳, **바닥**에 도착해요.

R² — 평균만 찍는 모델보다 얼마나 나은가



01 기준선은 '바보 모델'

아무 정보 없이 **무조건 평균**만 답하는 모델을 바닥에 깔아둡니다. 우리 모델이 이보다 나은 게 없다면 만들 이유가 없죠.

02 어긋남을 둘로 쪼갬다

점이 평균에서 벗어난 전체(SST) = 회귀선이 설명해 낸 몫(SSR) + 여전히 남은 오차(SSE). $SST = SSR + SSE$ 가 항상 성립해요.

03 $R^2 = SSR / SST = 1 - SSE / SST$

전체 변동 중 모델이 설명한 **비율**입니다. 설명 못 한 오차 비율(SSE/SST)을 1에서 뺀 값과 같죠. 1이면 완벽, 0이면 바보 모델과 동급 — 음수면 평균만도 못하다는 뜻이에요.

04 MSE와 다른 쓸모

MSE는 단위가 '가격의 제곱'이라 크기만 봐선 감이 안 옵니다. R²는 **0~1 비율**이라 데이터가 달라도 비교가 돼요 — 보고서의 단골 지표인 이유입니다.

한 점의 어긋남(SST)은 모델이 설명한 몫(SSR)과 못 잡은 오차(SSE)로 정확히 쪼개진다.

단서 하나로는 부족할 때 — 다중 선형회귀

면적만 보고 가격을 맞히는 데는 한계가 있습니다. 역세권, 층수, 연식... **단서를 여러 개** 쓰면 어떻게 될까요?

식은 그대로 늘어납니다. $y = w_1x_1 + w_2x_2 + \dots + b$ — 변수마다 자기 계수 w 를 하나씩 거느릴 뿐, '계수 곱해서 더하기'라는 뼈대는 단순 회귀와 똑같아요.

그림만 한 차원 올라갑니다. 변수 2개면 점들이 3차원 공간에 떠 있고, 선 대신 **평면**을 끼워 맞춥니다. 3개부터는 그림으로 못 그리는 **초평면**이 되지만 — 수식과 코드는 눈 하나 깜짝 안 해요.

학습법은 통째로 재활용입니다. MSE로 채점하고 경사를 내려가는 루프가 변수 수와 무관하게 그대로 돕니다. w 가 2개든 수십 개든 **각자 자기 편미분을 따라** 동시에 한 걸음씩 움직일 뿐이에요.

그래서 **현실 문제 대부분이 다중 회귀**입니다. 집값은 면적 하나가 아니라 위치·연식·층수가 **함께** 정하니까요. 다음 두 장에서 단순과 무엇이 같고 다른지, 그리고 학습된 계수를 어떻게 읽는지 봅니다.

단서 하나냐 여럿이냐 — 단순 vs 다중

두 회귀는 **계수 곱해 더하기**라는 뼈대가 같습니다. 변수 수와 그림의 차원만 달라져요.

변수 1개

단순 선형회귀

식 : $y = wx + b$

단서 하나(면적)로 가격을 맞춥니다. 찾을 미지수는 **w, b** 둘뿐이에요.

그림 : 2차원 직선

점들 사이로 **직선** 하나. 가로-세로 평면이라 학습 전 과정을 눈으로 볼 수 있죠.

언제 쓰나

관계가 거의 한 변수로 설명될 때, 그리고 **개념을 익히는 출발점**으로 좋습니다.

변수 여러 개

다중 선형회귀

식 : $y = w_1x_1 + w_2x_2 + \dots + b$

단서마다 계수 w를 하나씩. 미지수가 **변수 수만큼** 늘 뿐 뼈대는 같아요.

그림 : 평면 → 초평면

변수 2개면 3차원 속 **평면**, 그 이상은 그림으로 못 그리는 **초평면**. 수식·코드는 그대로예요.

언제 쓰나

현실 문제 대부분. 가격은 면적·위치·연식이 **함께** 정하니깐요.

계수는 영향력 보고서 — w를 읽는 법

학습이 끝나면 w들이 "어떤 단서가 결과를 어느 쪽으로 미는지"를 숫자로 말해줍니다. 그 숫자를 오해 없이 읽는 법.

부호 = 방향. $w_{\text{면적}} > 0$ 이면 면적이 클수록 가격↑, $w_{\text{연식}} < 0$ 이면 오래될수록 가격↓. 부호 하나로 그 단서가 결과를 **미는 방향**을 읽습니다.

크기 = 세기, 단 단위에 주의. 계수가 클수록 그 변수 1단위 변화가 가격을 많이 움직입니다. 하지만 면적(평)과 거리(m)는 **단위가 달라**, 계수 크기를 그대로 비교하면 안 돼요 — 비교하려면 변수를 같은 척도로 **표준화**한 뒤 봐야 합니다.

'다른 조건이 같을 때'가 숨은 전제. 다중 회귀의 계수는 **나머지 변수를 고정한 채** 그 변수만 변할 때의 효과입니다. 그래서 단순 회귀의 기울기와 값이 다를 수 있어요 — 다른 변수가 흡수하던 영향이 분리되니까요.

상관된 단서는 조심. 면적과 방 개수처럼 함께 움직이는 변수는 영향력이 둘로 갈려 계수가 불안정해질 수 있습니다. 이때 **규제**(§4)가 계수를 안정시켜 줘요 — 다음 절로 이어지는 이유입니다.

공부한 문제로 시험 보면 안 되니까

데이터를 **훈련용(train)**과 **평가용(test)**으로 가릅니다. 모델은 train만 보고 배우고, test는 **시험 당일까지 봉인** — 점수의 의미를 지키는 최소한의 장치예요.



01 전부로 배우고 전부로 시험 보면

문제집을 통째로 외운 학생과 같습니다. 100점이 나와도 **암기인지 실력인지** 구분이 안 돼요.

02 test = 미래의 대역

모델의 존재 이유는 **처음 보는 데이터** 예측입니다. 봉인해 둔 test가 그 '새 데이터' 역할을 대신해요.

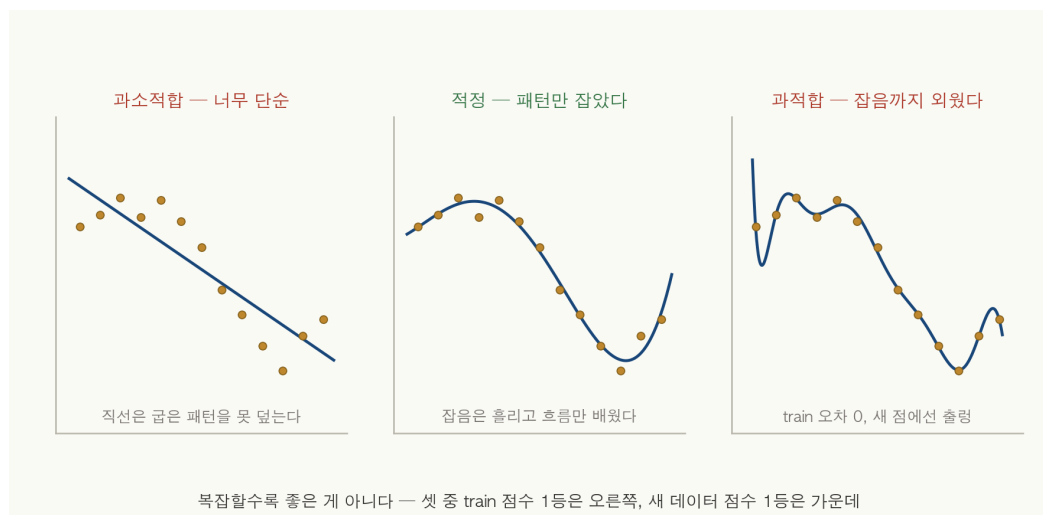
03 8 : 2가 무난한 출발

배울 양은 넉넉히, 시험은 믿을 만큼만. 데이터가 아주 크면 test 비율을 더 줄이기도 합니다.

04 셔플 후 random_state 고정

정렬된 데이터를 그냥 자르면 한쪽에 쏠려요. **섞은 뒤** 나누고, 숫자를 고정해 누구나 같은 분할을 얻습니다.

더 구불구불한 선이 항상 더 좋을까



같은 데이터, 세 가지 복잡도 — train 점수 1등은 오른쪽이지만, 새 데이터 점수 1등은 가운데다.

01 과소적합 — 너무 단순

굽은 패턴을 직선 하나로 덮으면 train에서도 test에서도 못 맞춥니다. 모델의 **표현력이 모자란** 상태예요.

02 과적합 — 너무 열심히

고차 다항식은 모든 점을 **관통**할 수 있습니다. train 오차는 거의 0 — 그런데 점과 점 사이에선 미친 듯이 출렁여서, 새 데이터 앞에서 무너져요.

03 외운 것이지 배운 게 아니다

잡음(우연한 출렁임)까지 정답으로 외운 상태입니다. **앞 장에서 test를 따로 떼어 둔 진짜 이유** — 과적합은 test 점수에서만 들통나거든요.

04 처방은 두 갈래

모델을 단순하게 바꾸거나(차수 낮추기), 복잡한 모델을 두되 **날뛰지 못하게 묶거나**. 두 번째 방법이 규제 — 다음 장입니다.

벌점으로 묶는다 — 규제 Regularization

비용 함수에 **계수 크기에 대한 벌점**을 더합니다 — Lasso는 $\text{MSE} + \alpha \cdot \sum |w|$ (L1), Ridge는 $\text{MSE} + \alpha \cdot \sum w^2$ (L2). 오차도 줄여야 하고 계수도 못 키우니, 모델은 꼭 필요한 만큼만 복잡해져요.

01 과적합의 증상 = 큰 계수

출렁이는 선의 원인은 +1500, -2300처럼 **비정상적으로 큰 계수**입니다. 계수를 억제하면 선이 완만해져요.

02 L1 (Lasso) — 다이아몬드

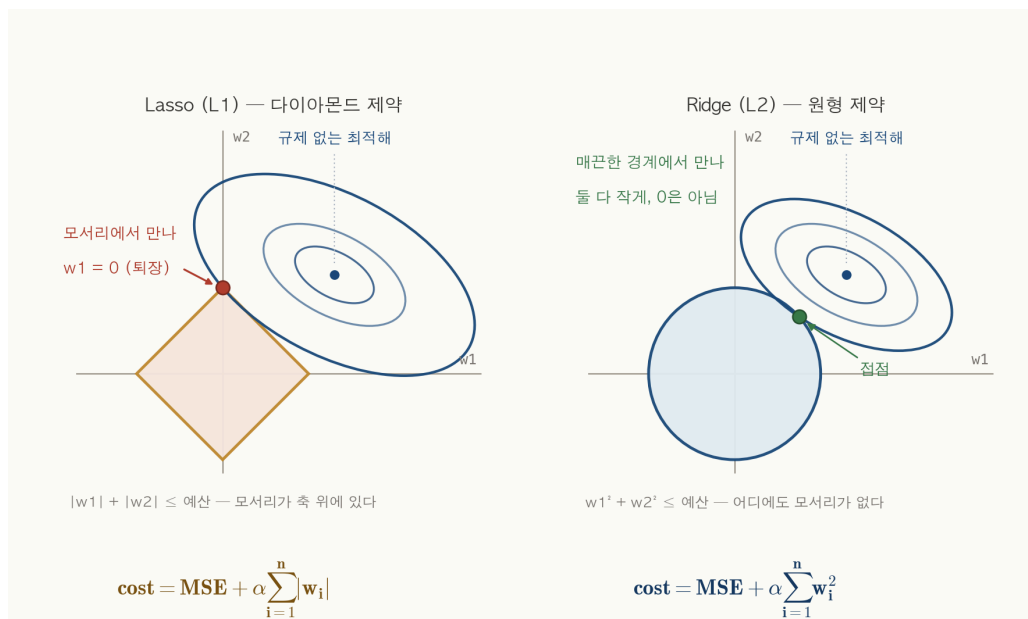
벌점이 $|w|$ 의 합. 제약 영역에 **모서리**가 있어 최적해가 자주 거기 닿고 — 그 자리의 계수는 정확히 0이 됩니다.

03 L2 (Ridge) — 원

벌점이 w^2 의 합. 영역이 둥글어 모서리가 없으니 계수가 0이 되진 않고, **전원이 고르게** 작아져요.

04 α — 벌점의 세기

클수록 모델이 단순해지고(과소적합 위험), 0이면 규제 없는 회귀로 복귀합니다. **과소와 과적 사이**를 α 로 조준해요.



Lasso vs Ridge — 자르느냐, 줄이느냐

같은 '계수 누르기'지만 결과의 성격이 다릅니다. 데이터에 대한 믿음에 따라 골라요.

L1 — 계수를 0으로 자른다

Lasso

일부 피처를 꺼버린다

벌점을 건디지 못한 계수는 정확히 0 — 그 피처는 모델에서 퇴장합니다. 살아남은 피처만 보면 돼요.

공짜 피처 선택

수백 개 피처 중 진짜 중요한 소수만 남기고 싶을 때 강력합니다. 모델이 단순해지니 해석도 같이 쉬워지죠.

이럴 때

"피처 대부분은 사실 무관할 것"이라 믿을 때 — 잡다한 단서가 많이 섞인 데이터에 어울립니다.

L2 — 전체를 고르게 줄인다

Ridge

아무도 퇴장시키지 않는다

계수가 0엔 닿지 않고 모두 조금씩 작아집니다. 모든 단서의 말을 조금씩은 계속 듣는 셈이에요.

많은 피처에 강하다

방 수와 면적처럼 상관 높은 피처들 앞에서 Lasso는 하나만 남기고 버리지만, Ridge는 영향력을 나눠 갖게 해 안정적입니다.

이럴 때

"피처 대부분이 조금씩은 기여할 것"이라 믿을 때 — 고민되면 Ridge부터 써보는 게 무난한 기본기입니다.

정리 — 회귀의 한 사이클

오늘 한 일

- ▶ **회귀** — 연속값 예측, 독립변수 X 로 종속변수 y 를 맞힌다
- ▶ **$y = wx + b$** — 채점은 MSE, 수정은 경사하강법, 보폭은 α
- ▶ **R^2** — 평균만 찍는 바보 모델 대비 설명력, 1에 가까울수록 좋다
- ▶ **다중 회귀** — 단서를 여러 개로, 학습된 계수가 곧 각 단서의 영향력
- ▶ **규제** — Lasso(L1)는 계수를 0으로, Ridge(L2)는 고르게 축소

선을 긋고(가설), 오차를 재고(MSE), 경사하강법으로 계수를 고친다. 단서가 많아지면 다중 회귀로 넓히고, 평가용 데이터는 따로 떼어 두며, 과적합은 규제로 억제한다. 이 한 사이클이 이후 대부분의 머신러닝에서 반복된다.

더 나아가려면

- ▶ **직접 돌려보기** — sklearn `LinearRegression`으로 데이터를 예측
- ▶ **계수로 해석** — w 의 부호·크기로 각 단서가 결과를 미치는 방향을 읽기
- ▶ **과적합 점검** — train·test 점수를 비교, 벌어지면 규제(Lasso·Ridge)
- ▶ **평가 넓히기** — R^2 외에 MAE·RMSE 등 오차 지표도 함께 보기
- ▶ **범주를 맞춰야 하면** — 로지스틱 회귀 같은 분류 모델로